

```
1 """
2 Nombre completo: Merin Zepeda Esteban Gabriel
3 Grupo: 951
4 Fecha de realización: 08/02/2026
5 Descripción: Solución de tres ejercicios:
6 1. Cree una clase llamada Estadística que contiene
   como atributo una lista de números naturales la cual
   puede contener repetidos.
7 2. Simula el historial de cambios en una hoja de
   cálculo, donde los usuarios pueden realizar cambios
   en las celdas.
8 3. Simulador de navegación de robot en almacén con
   recogida de productos
9 """
10
11
12 # EJERCICIO 1: ESTADÍSTICA BÁSICA
13
14 """
15 Descripción del problema:
16 Crear una clase Estadística que analice una lista de
   números naturales,
17 calculando frecuencias, moda y generando histogramas
   visuales.
18 """
19
20
21 class Estadistica:
22     def __init__(self, lista):
23
24         self.lista = lista
25
26     def frecuencia_numeros(self):
27         frecuencias = {}
28
29         for numero in self.lista:
30             if numero in frecuencias:
31                 frecuencias[numero] += 1
32             else:
33                 frecuencias[numero] = 1
34
```

```
35         lista_tuplas = [(num, frec) for num, frec in
    frecuencias.items()]
36         lista_tuplas.sort()
37
38         return lista_tuplas
39
40     def moda(self):
41         frecuencias = self.frecuencia_numeros()
42
43         max_frecuencia = 0
44         numero_moda = None
45
46         for numero, frecuencia in frecuencias:
47             if frecuencia > max_frecuencia:
48                 max_frecuencia = frecuencia
49                 numero_moda = numero
50
51         return numero_moda
52
53     def histograma(self):
54         frecuencias = self.frecuencia_numeros()
55
56         for numero, frecuencia in frecuencias:
57             asteriscos = '*' * frecuencia
58             print(f"{numero} {asteriscos}")
59
60
61 # Ejemplo de uso ejercicio 1
62 print('Ejercicio 1')
63 lista = Estadistica([1, 3, 2, 4, 2, 2, 3, 2, 4, 1, 2
    , 1, 2, 3, 1, 3, 1])
64 print(f'Veces repetido:{lista.frecuencia_numeros()}')
65 print(f"Moda: {lista.moda()}")
66 print("Histograma:")
67 lista.histograma()
68 print()
69
70
71 # EJERCICIO 2: HISTORIAL DE CAMBIOS EN HOJA DE
    CÁLCULO
72
```

```

73 """
74 Descripción del problema:
75 Simular un sistema de historial de cambios en una
    hoja de cálculo,
76 permitiendo registrar cambios en celdas y
    deshacerlos usando una pila.
77 """
78
79
80 def registrar_cambio(historial_cambios, celda,
    valor_anterior):
81     historial_cambios.append((celda, valor_anterior
    ))
82
83
84 def deshacer_cambio(historial_cambios):
85     if len(historial_cambios) > 0:
86         cambio_deshecho = historial_cambios.pop()
87         return cambio_deshecho
88     else:
89         return None
90
91
92 # Ejemplo de uso ejercicio 2
93 print('Ejercicio 2')
94 historial_cambios = []
95 registrar_cambio(historial_cambios, 'A1', 10)
96 registrar_cambio(historial_cambios, 'B2', 20)
97 print(f'cambios realizados: {historial_cambios}')
98 deshacer_cambio(historial_cambios)
99 print(f'Deshacer cambios:{historial_cambios}')
100 print()
101
102 # EJERCICIO 3: NAVEGACIÓN EN ALMACÉN
103
104 """
105 Descripción del problema:
106 Simular un robot que navega por un almacén
    representado como cuadrícula,
107 recogiendo productos (P) y evitando obstáculos (#),
    con retorno al inicio.

```

```

108 Movimientos: R (derecha), D (abajo), L (izquierda),
    U (arriba)
109 """
110
111
112 def verificar_recogida_productos(almacen,
    movimientos):
113     """
114         Verifica si los movimientos permiten recoger
todos los productos
115         y retornar al punto de inicio
116         :param almacen: Matriz que representa el almacén
117         :param movimientos: Lista de movimientos ['R', 'D', 'L', 'U']
118         :return: True si los movimientos son válidos,
False en caso contrario
119     """
120     fila_actual = 0
121     columna_actual = 0
122     print('Ejercicio 3')
123     print(f"Posición inicial del robot: ({fila_actual}, {columna_actual})")
124
125     productos_totales = 0
126     for fila in almacen:
127         for celda in fila:
128             if celda == 'P':
129                 productos_totales += 1
130
131     productos_recogidos_set = set()
132
133     filas = len(almacen)
134     columnas = len(almacen[0]) if filas > 0 else 0
135
136     for i, movimiento in enumerate(movimientos):
137         nueva_fila = fila_actual
138         nueva_columna = columna_actual
139
140         if movimiento == 'R': # Derecha
141             nueva_columna += 1
142         elif movimiento == 'D': # Abajo

```

```

143         nueva_fila += 1
144         elif movimiento == 'L': # Izquierda
145             nueva_columna -= 1
146         elif movimiento == 'U': # Arriba
147             nueva_fila -= 1
148         else:
149             print(f"Movimiento {i+1}: {movimiento}
- Movimiento inválido")
150             return False
151
152         if nueva_fila < 0 or nueva_fila >= filas or
nueva_columna < 0 or nueva_columna >= columnas:
153             print(f"Movimiento {i+1}: {movimiento}
- Fuera de límites en ({nueva_fila}, {nueva_columna
})")
154             return False
155
156         if almacen[nueva_fila][nueva_columna] == '#'
:
157             print(f"Movimiento {i+1}: {movimiento}
- ¡Obstáculo encontrado en ({nueva_fila}, {
nueva_columna})!")
158             return False
159
160         fila_actual = nueva_fila
161         columna_actual = nueva_columna
162
163         if almacen[fila_actual][columna_actual] == '
P':
164             posicion_producto = (fila_actual,
columna_actual)
165             if posicion_producto not in
productos_recogidos_set:
166                 productos_recogidos_set.add(
posicion_producto)
167                 print(f"Movimiento {i+1}: {
movimiento} -> Posición ({fila_actual}, {
columna_actual}) - ¡Producto recogido! ({len(
productos_recogidos_set)}/{productos_totales})")
168             else:
169                 print(f"Movimiento {i+1}: {

```

```
169 movimiento} -> Posición ({fila_actual}, {
    columna_actual}))")
170         else:
171             print(f"Movimiento {i+1}: {movimiento}
    -> Posición ({fila_actual}, {columna_actual}))")
172
173         if fila_actual == 0 and columna_actual == 0:
174             if len(productos_recogidos_set) ==
productos_totales:
175                 print(f"Todos los productos han sido
recogidos ({len(productos_recogidos_set)}/{
productos_totales}) t el robot regresó al inicio.")
176                 return True
177             else:
178                 print(f"El robot regresó al inicio, pero
solo recogió {len(productos_recogidos_set)}/{
productos_totales} productos.")
179                 return False
180
181
182 almacen = [
183     ['.', '.', '#', 'P'],
184     ['.', '#', '.', '.'],
185     ['P', '.', 'P', '.'],
186     ['#', '.', '#', '.']
187 ]
188 movimientos_correctos = ['D', 'D', 'R', 'R', 'U', 'R
', 'U', 'D', 'L', 'D', 'L', 'L', 'U', 'U']
189 print(verificar_recogida_productos(almacen,
    movimientos_correctos))
```